

**Introduction to SQL**

- SQL - introduction , advantages and disadvantages
- Data types – built-in (number, char, varchar2, date, raw, long raw)
- Types of SQL Statements : DDL (Data Definition Language), DML (Data Manipulation Language), DCL (Data Control Language), TCL (Transaction Control Language)
- Working with SQL\*Plus – overview and basic commands like ed, start, get, save, exit, connect, set linesize, set pagesize and host
- Creating table and inserting data - CREATE TABLE, INSERT, retrieving data using query – SELECT , manipulating data – DELETE and UPDATE , modifying and removing table – ALTER TABLE and DROP TABLE

**SQL :**

SQL is the language used to communicate with the server to access, manipulate and control data.

❖ **Features of SQL:**

- Can be used by a range of users, including those with little or no programming experience.
- Is a non-procedural language.
- Reduces the amount of time required for creating and manipulating systems.
- Is an English like language.
- SQL commands may be entered on one or many lines
- Clauses are usually placed on separate lines for readability and ease of editing.
- Tabs and indents can be used to make code more readable.
- Command words cannot be split across line or abbreviated.
- SQL statements are not case sensitive.
- An SQL command entered at the SQL prompt, and subsequent lines are numbered. This is called the SQL buffer.
- Statement can be executed in a number of ways.
  - a. Place a semicolon (;) at the end of last clause
  - b. Place a semicolon or slash on the last line in the buffer.
  - c. Place a slash at the SQL prompt.
  - d. Issue a SQL\*Plus START command at the SQL prompt.

❖ **Advantages of SQL :****Standard independent language:**

The widespread rules of the SQL have been recognized by ANSI and ISO. Therefore the SQL is an open language, involving it is not owned or controlled by a single company. Nowadays, SQL is offered by all the foremost DBMS vendors. SQL is not just for an exacting product; it also works with oracle, Microsoft SQL server and Sybase, and lot more. Almost all the data base vendors may have proprietary extensions to SQL, but the fundamentals of SQL are almost equal across all the data base vendors.

**Cross-platform abilities:**

The prevalent adversity of using a programming language to access database is that it infrequently produces an accurate cross platform application. SQL may be moderately innovative to some of the for the most part of popular programming languages like COBOL or C. But it has been in use on dissimilar hardware platforms for years. Majority of the cases, the same SQL statement can be used on a mainframe, desktop and a server.

**Easy to learn and use:**

SQL statements look like very simple English sentences that are easy to understand. SQL has been created such that it is instinctive, easy, and maps human's cognitive model. While it is non-procedural in nature, the user host has to type SQL declaration and afterwards it has to be handed over to the DBMS. Then it is executed by the DBMS, internal code and returns a set, which is a logically defined group of data.

**Speed:**

For the past ten years, SQL data engines have been focusing the powerful effort to get better performance. The passionate competition among data base vendors has resulted in quicker, more vigorous database management systems that work at lower costs.

❖ **Disdvantages of SQL :**

SQL cannot be used for programming because SQL does not provide the programming techniques of condition checking , looping and branching etc.

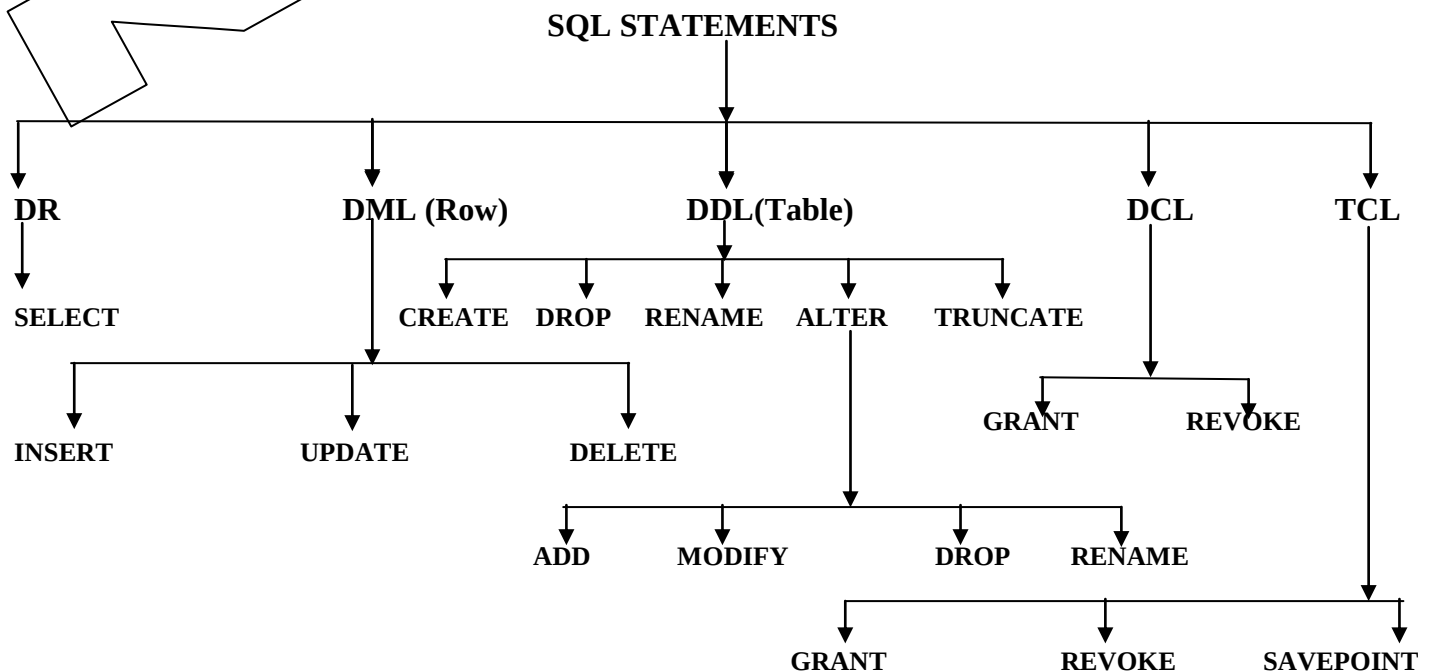
SQL statements are passed to oracle engine one at a time which increases the traffic on the network which results in decrease of speed of data processing.

**DATA TYPE**

1. **Varchar 2(size):** variable length character data(min-1, max-4000)
2. **CHAR (size):** Fixed length character data(min-1, max-2000)
3. **NUMBER (P, S):** Variable length numeric data(precision- 1 to 38 and scale- 84 to 127).
4. **DATE:** Date and time values.
5. **LONG:** Variable length character data up to 2 gigabytes.
6. **RAW and LONG RAW:** Raw binary data.

**TYPES OF SQL STATEMENT :**

- **Data Retrieval:** SELECT
- **Data Manipulation Language:** INSERT, UPDATE, DELETE, MERGE.
- **Data Definition Language:** CREATE, ALTER, DROP, RENAME, TRUNCATE.
- **Transaction Control:** COMMIT, ROLLBACK, SAVE POINT.
- **Data Control Language:** GRANT, REVOKE.



**Data Definition Language :** Data definition language (DDL) statements let you to perform the tasks like Create, alter, and drop schema objects. Oracle Database implicitly commits the current transaction before and after every DDL statement.

**Data Manipulation Language :** Data manipulation language (DML) statements access and manipulate data in existing schema objects. These statements do not implicitly commit the current transaction.

**Data Control Language :** You can issue native data control language (DCL) statements from an Oracle environment, allowing central administration of user privileges and access levels for heterogeneous data stores.

**Transaction Control Language :** Transaction control statements manage changes made by DML statements.

## WORKING WITH SQL\*PLUS :

### ❖ Features of SQL\*PLUS:

- Accepts adhoc entry of statements.
- Accepts SQL and PL/SQL input from files.
- Edits SQL statements with a line editor.
- Controls Environmental Settings.
- Formats query results into basic reports.
- Interact with end users.
- Accesses remote database.

### ❖ Basic SQL\*PLUS Commands:

|              |                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------|
| Ed           | :Load the SQL * PLUS buffer into an editor. By Default, saves the file to AFIEDT.BUF (ED <filename> / Path) |
| Start        | :Run an SQL Script (START <filename.sql> / Path)                                                            |
| Get          | :Retrieve the previously stored command file (Get <Filename>)                                               |
| Save         | :Save the contents of the SQL*PLUS buffer in a command file (Save <filename>)                               |
| Exit         | :Commit, logoff and exit                                                                                    |
| Connect      | :connect to a database as a specified user.<br>(Connect username/password@SID)                              |
| Set linesize | :display or change SQL * PLUS line size setting.                                                            |
| Set pagesize | : display or change SQL * PLUS page size setting.                                                           |
| Host         | : Execute host operating system command (HOST CD Scripts)                                                   |

## DATA RETRIEVAL:-

### ❖ SELECT statements:

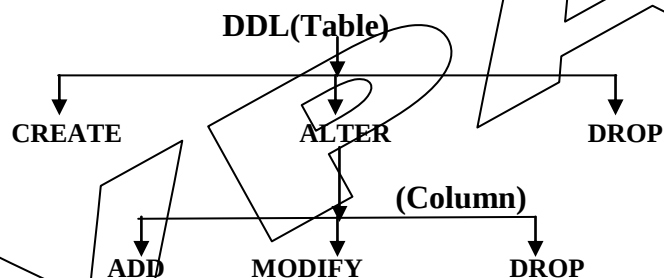
A SELECT statement retrieves information from the database.

1. **Projection:** You can use the projection capability in SQL to choose the columns in a table that you want returned by your query.
2. **Selection:** To choose a row in a table that you want returned by your query.
3. **Joining:** To bring together data that is stored in different tables by creating a link between them.

**SYNTAX:**

SELECT  
 \*  
 Distinct  
 Column/Expression  
 Alias  
 From Table  
 Where  
 Order By

You can display all columns of data in a table by following the select keyword with an asterisk (\*).

**DDL :**

- The CREATE TABLE statement:**

Create tables to store data by executing the SQL CREATE TABLE statement. This statement is one of the data definition language (DDL) statements. DDL statements are subset of SQL statements used to create, modify or remove database structure. These statements have an immediate effects on the database, and they also record information in the data dictionary.

To create a table, a user must have the CREATE TABLE privilege and a storage area in which to create object.

| SYNTAX                                                                                                                                                                          | EXAMPLE                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| CREATE TABLE <table name><br>(column1 data type <(size)> default <value>,<br>. . . default <value>,<br>. . . default <value>,<br>Column N <datatype> <(size)> default <value>); | CREATE TABLE emp<br>( Eno number(5),<br>Ename varchar2(15),<br>Design varchar2(15) default clerk,<br>Salary number(7,2)); |

**Default Statement :-** value followed by DEFAULT key word will be automatically inserted in the table if no value will be inserted at the time of insertion record in the table.

- The Alter Table Statement:**

**Use the Alter Table Statement to:**

1. Add a new column.
2. Modify an existing column.
3. Define a default value for the new column.

4. Drop a column.
5. You can Rename a column by using Alter table statement.

You can add, modify, drop and rename columns to a table by using the Alter Table statement.

❖ **Adding a column:**

| SYNTAX                                                                | EXAMPLE                                      |
|-----------------------------------------------------------------------|----------------------------------------------|
| ALTER table <Table Name><br>ADD (<Column name> <datatype> <(size)> ); | ALTER table Emp<br>ADD ( Comm number(6,2) ); |

**Guideline:**

1. You can add or modify columns.
  2. You cannot specify where the column is to appear. The new column becomes the last column.
- **Note:** If a table already contains rows when a column is added, then the new column is initially null for all the rows.

❖ **Modifying a column:**

Column modification can include changes to a column data type; size and default value.

| SYNTAX                                                                                    | EXAMPLE                                                  |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------|
| ALTER table <Table Name><br>MODIFY column (<Column name><br><new datatype> <new (size)>); | ALTER table Emp<br>MODIFY column ( Ename varchar2(25) ); |

**Guideline:**

1. You can increase the width or precision of numeric column and also width of character column.
2. You can decrease the width of a column only if the column contains only null-values or if the table has no rows.
3. You can change the data type only if the column contains null values.
4. You can convert a CHAR column to VARCHAR2 data type or convert a VARCHAR2 column to the CHAR data type only if the column contains null values or if you do not change the size.
5. A change to the default value of a column affects only subsequent insertion to the table.

❖ **Dropping a column:**

You can drop a column from a table by using Alter Table.

| SYNTAX                                                  | EXAMPLE                              |
|---------------------------------------------------------|--------------------------------------|
| ALTER table <Table Name><br>DROP column <Column name> ; | ALTER table Emp<br>DROP column comm; |

**Guideline:**

1. The column may or may not contain data.
2. Using a Alter Table statement, only one column can be dropped at a time.
3. The table must have at least one column remaining in it after it is altered.
4. Once a column is dropped, it cannot be recovered.

❖ **DROPPING a Table:**

The drop table statement removes the definition of an oracle table. When you drop a table, the database loses all the data in the table and all the indexes associated with it.

| SYNTAX                   | EXAMPLE         |
|--------------------------|-----------------|
| DROP TABLE <Table name>; | DROP TABLE emp; |

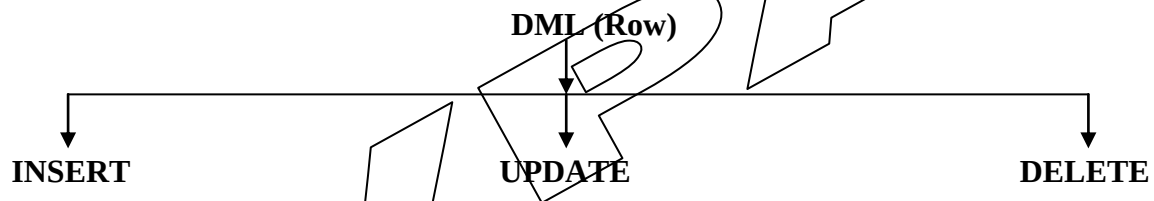
**Guideline:**

1. All data is deleted from the table.
2. Any views and synonyms are remain but are invalid.
3. Any pending Transaction are committed.
4. Only the creator of the table as user with the DROP ANY TABLE privilege can remove a table.

**Note:** The DROP TABLE statement once executed is irreversible.

**MANIPULATING DATA**❖ **Data Manipulation Language:**

DML is a core part of SQL. When you ADD, UPDATE or DELETE data in the database, you execute a DML statement. A collection of DML statements that form a logical unit of work is called a transaction.

❖ **Adding a new Row to a Table:**

You can add new rows to a table by issuing the INSERT statement.

| SYNTAX                                                                | EXAMPLE                                                           |
|-----------------------------------------------------------------------|-------------------------------------------------------------------|
| INSERT into <table name><br>VALUES (val1, val2, val3, . . . valN);    | INSERT into Emp<br>Values ( 1, 'XYZ', 'CLERK', 10000);            |
| INSERT into <tablename> (col1, col2)<br>VALUES (col1value, col2value) | INSERT into Emp (Eno, Ename, Salary)<br>Values (1, 'PQR', 25000); |

**NOTE:** This statement with the values clause adds only one row at a time to a table.

1. Because you can insert a new row that contains values for each column, the column list is not required in the INSERT clause. However, if you do not use the column list, the values must be listed according to the default order of the columns in the table, and a value must be provided for each column.
2. Enclose character and date values within single quotation marks it is not recommended to enclose numeric values within single quotation marks.
3. Number values should not be enclosed in single quotes, because implicit conversion may take place for numeric values assigned to NUMBER data type columns if single quotes are included.

❖ **Methods for Inserting Null Values.**

1. **Implicit:** Omit the column from the column list.
2. **Explicit:** Specify the NULL keyword in the values list, specify the empty string ( ' ') in the VALUES list for character strings and dates.  
The Oracle server automatically enforces all data types, data ranges and data integrity constraints. Any column that is not listed explicitly obtains a null value in the new row.

❖ **Common errors that can occur during user input:**

- Mandatory value missing for a NOTNULL column.
- Duplicate value violates uniqueness constraint.
- Foreign key constraint violated.
- CHECK constraint violated.
- Data type mismatch.
- Value too wide to fit in column.

You can use SQL functions to enter special values in your table.

You can also use the USER function when inserting rows in the table. The user functions records the current username.

#### ❖ Creating a Script:

- Use & substitution in a SQL statement if prompt for values.
- & is a place holder for the variable value.
- Use '&' Substitution for accepting a value for a column from the users.

You can save command with substitution variables to a file and execute the commands in the file.

| SYNTAX                                                                       | EXAMPLE                                                                       |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| INSERT into <table name><br>VALUES (&col1,&col2, &col3,.. . . &colN)<br>/    | INSERT into Emp<br>Values ( &eno, '&Ename', '&desig', &salary)<br>/           |
| INSERT into <tablename> (col1, col2)<br>VALUES (&col1value, &col2value)<br>/ | INSERT into Emp (Eno, Ename, Salary)<br>Values (&eno, '&Ename', &salary)<br>/ |

#### ❖ UPDATE statement:

You can modify existing rows by using the UPDATE statement.

| SYNTAX                                                      | EXAMPLE                                         |
|-------------------------------------------------------------|-------------------------------------------------|
| UPDATE <Tablename><br>SET column= value<br>where condition; | UPDATE emp<br>SET salary =20000<br>Where eno=2; |

1. The UPDATE statement modifies specific rows if the WHERE clause is specified. If you omit the WHERE clause, all the rows in the table are modified.
2. You can update multiple columns in the SET clause of an UPDATE statement by writing multiple subqueries.

**Note:** If NO rows are updated, a message “0 rows updated” is returned

#### ❖ The DELETE statement:

You can remove existing rows by using the DELETE statement.

| SYNTAX                                       | EXAMPLE                                 |
|----------------------------------------------|-----------------------------------------|
| DELETE FROM <Table Name><br>where condition; | DELETE from Emp<br>where salary = 10000 |

**NOTE:** If no rows are deleted, a message “0 rows deleted” is returned.

You can delete specific rows by specifying the WHERE clause in the DELETE statement.

If you omit the WHERE clause, all rows in the table are deleted.

1. You can use subqueries to delete rows from a table based on the values from another table.
2. If you attempt to delete a record with a value that is tied to an integrity constraint.

### ❖ Single Ampersand Substitutional Values:

When running a report, users often want to restrict the data returned dynamically. SQL\*PLUS provide this flexibility by means of user variables. Use an ampersand (&) to identify each variable in your SQL statement. You do not need to define the value of each variable.

**& (user variable):** Indicates a variable in a SQL statement; if the variable does not exist, SQL\*PLUS prompts the user for a value (SQL\*PLUS) discards a new variable once it is used.

- With a single ampersand (&) we can accept a value from the user for **INSERT INTO <table name> VALUES** statement too.
- With a single ampersand, the user is prompted every time the command is executed, if the variable does not exist.
- When SQL\*PLUS detects that the SQL statement contains an &, you are prompted to enter a value for the substitution variable named in the SQL statement. Once you enter a value and click the submit for execution button, the results are displayed in the output area of your SQL\*PLUS session.

### • Specifying character and date values with substitution variables:

In a WHERE clause, date and character values must be enclosed within single quotation marks. The same rule applies to the substitution variables.

You can also use functions such as UPPER and LOWER with the ampersand. Use UPPER ('&Job\_title').

### • Specifying Column Names, Expressions and Text:

- Not only can you use the substitution variables in WHERE clause of SQL statement, but these variables can also be used to substitute for column names, expressions, or text.
- If you do not enter a value for the substitution variable, you will get an error when you execute the preceding statement.

### ❖ Creating a script file to run a report:

You can either enter each of the SQL\*PLUS commands at the SQL prompt or put all the commands including the select statement, in a command file. A typical script consists of a least one SELECT statement and several SQL\*PLUS commands.

### ❖ How to Create Script File:

1. Create the SQL select statement at the SQL prompt. Ensure that the data required for the report is accurate before you save the statement to a file and supply formatting commands. Ensure that the relevant ORDER By clause is included if you intend to use break.
2. Save the SQL statement to a script file.
3. Edit the script file to enter the SQL\*PLUS commands.
4. Add the required formatting command before the select statement. Be certain not to place SQL\*PLUS commands within the SELECT statement.



5. Verify that the SQL statement is followed by a run character, either a semicolon (;) or a slash (/).
6. Add the format clearing SQL\*PLUS commands after the run character. Alternatively you can store all the format clearing commands in a reset file.
7. Save the script file.
8. Execute the script file by using @ sign or writing command 'START' into the SQL\*PLUS text window.

❖ **Guideline:**

1. You can include blank lines between SQL PLUS commands in script.
2. If you have a lengthy SQL PLUS command you can continue it on the next line by ending the current line by (-).
3. You can abbreviate SQL\*PLUS commands.
4. Include reset commands at the end of the file to restore the original SQL PLUS environment.
5. REM represents a remark or comments in SQL PLUS.

SQL\*PLUS